System Design Interview: Design Airbnb

#113: System Design Interview

NEO KIM AND HAYK

JAN 15, 2026 PAID

A hotel booking system like Booking.com or Airbnb is a common system design interview question.

It looks simple, right until it breaks in the worst way.

Two users click “Book” on the last available room in the same second. Your API does a SELECT check, so both requests see availability, and you charge two credit cards for one room. Next, you’re dealing with refunds, angry customers, and a system that can’t be trusted.

And that’s not a bug… That’s the default outcome if you don’t design it correctly.

This is why Airbnb and Booking.com aren't just "tables + endpoints".

Most developers jump straight into boxes and arrows without understanding the actual problems. They miss concurrency[1] traps, forget double booking[2], and can't explain tradeoffs.

> In this newsletter, we'll design it the way interviews and real traffic demand: with strong consistency for inventory, fast reads for search, and clear tradeoffs for caching and scaling.

But before we design anything, let's zoom out, define the real problem, and lay out the approach step by step.

Onward.

Find out why 150K+ engineers read The Code twice a week (Partner)

That's exactly why 150K+ engineers working at Google, Meta, and Apple read The Code twice a week.

Here's what you get:

- **Curated tech news that shapes your career** - Filtered from thousands of sources so you know what's coming 6 months early.
- **Practical resources you can use immediately** - Real tutorials and tools that solve actual engineering problems.
- **Research papers and insights decoded** - We break down complex tech so you understand what matters.

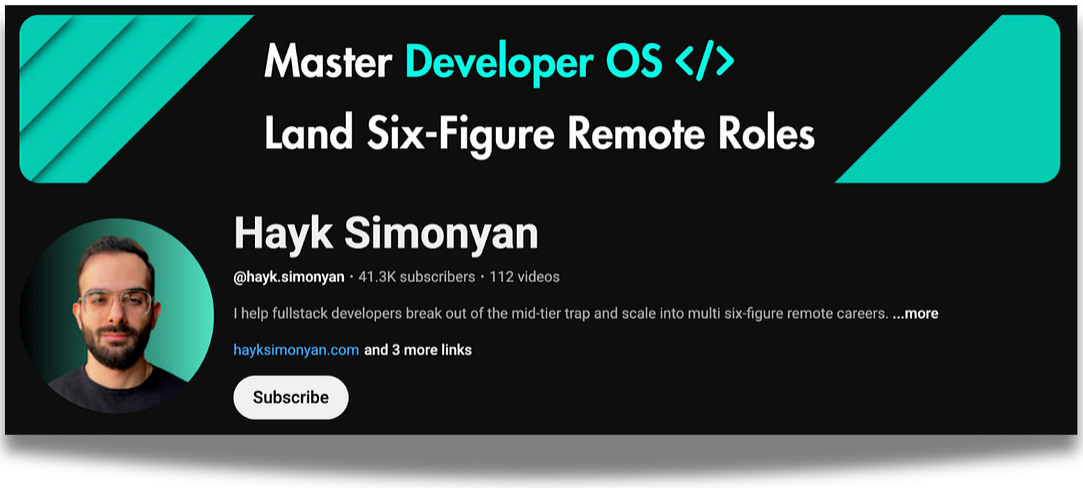
All delivered twice a week in just 2 short emails.

Sign up and get access to the Ultimate Claude code guide to ship 5X faster.

Join 150K+ Engineers

(Thanks for partnering on this post and sharing the ultimate claude code guide.)

I want to reintroduce Hayk Simonyan as a guest author.



He's a senior software engineer specializing in helping developers break through their career plateaus and secure senior roles.

If you want to master the essential system design skills and land senior developer roles, I highly recommend checking out Hayk's **YouTube channel**.

His approach focuses on what top employers actually care about: system design expertise, advanced project experience, and elite-level interview performance.

Let's dive in!

What You'll Learn

- Design Requirements - How to scope the system
- Back of the Envelope Calculations[3] - Math that proves you understand scale
- High-Level Design - System components and architecture

- API Design - RESTful endpoints
- Data Model - Schema decisions that matter
- Handling Concurrency - Real approaches that work in production
- Scaling the System - What happens when you hit real traffic
- Handling Failures and Edge Cases - Problems that break systems

---

Design Requirements

Every well-architected system design starts with constraints.

If you skip this step, everything that follows is guesswork.

Clarifying Questions

Here are some clarifying questions we'll ask about the design requirements.

- **Scale**:
  - How many hotels?
  - How many rooms in total?
  - Single chain or platform like Booking.com?
- **Payment timing**:
  - Pay at booking or check-in?
- **Booking channels**:
  - Web only?
  - Mobile apps?
  - Phone reservations?
- **Cancellation**:
  - Can users cancel?

    - What's the policy?
- **Overbooking**[4]:
    - Do we support it? (Hotels typically allow 5-10% overbooking)
- **Dynamic pricing**:
    - Does the room price change based on demand/date?

With these constraints clarified, we can now define what the system must actually do.

Functional Requirements

- **Hotel Management**: Admins can add, update, and remove hotels and room types. They adjust pricing and inventory[5].
- **Search & Discovery**: Users can search for hotels by location, dates, guest count, price range, and amenities.
- **Reservation Flow**: Users can book rooms, view reservations, and cancel bookings. The system should prevent double bookings.
- **Payment Processing**: Integrate with payment gateways. And handle failures gracefully.
- **Notifications**: Send confirmations for bookings, cancellations, and payments.

Functional requirements define behavior, while non-functional requirements determine whether the system can withstand real traffic.

Non-Functional Requirements

- **Strong Consistency**: Two users cannot book the same room for the same dates. Non-negotiable.
- **High Concurrency**: Handle thousands of users trying to book the same hotel simultaneously.
- **Low Latency**: Search results under 500ms. Booking confirmation in 2-3 seconds.

- **High Availability**: 99.9%+ uptime.
- **Scalability**: Scale horizontally without redesigning the system.

Now that we know what the system must support, let's start with back-of-the-envelope calculations.

Back of the Envelope Calculations

Booking Holdings reported over 1.1 billion room nights[6] in 2024.

Let's turn that into usable numbers:

Assumptions

- Room nights per year: 1.1 billion
- Average stay: 2 nights per booking
- Days per year: 365

Daily Reservations and TPS

Bookings per year:

- 1.1B room nights / 2 nights per booking ≈ 550M bookings/year

Bookings per day:

- 550M / 365 ≈ 1.5M bookings/day

Bookings per second (TPS[7]):

- 1.5M / 86,400 ≈ 17 bookings/second

At Booking.com scale:

- ~1.5 million bookings per day

- ~17 bookings per second on average

Traffic Funnel and Read QPS

Similarweb shows ~516.5 million visits per month to booking.com with about 7.56 pages per visit.

Visits per day:

- 516.5M / 30 ≈ 17M visits per day

Visits per second:

- 17M / 86,400 ≈ 200 visits per second

If we assume around 5 backend reads per visit (search queries, availability checks, pricing):

- Backend read QPS[8] ≈ 200 visits/s × 5 ≈ 1,000 read requests per second

Key insights:

- This is strongly read-heavy.
- You handle around 1,000 read QPS and only ~17 write TPS.
- Your architecture should optimize for reads while keeping writes consistent.

Storage Estimation

For the inventory table, tracking availability by room type and date.

Assume:

- Average hotel occupancy: 70%
- Room nights sold per year: 1.1B

Total annual capacity:

- 1.1B / 0.7 ≈ 1.57B room nights/year

Total rooms on the platform:

- 1.57B / 365 ≈ 4.3M rooms

If an average hotel has 50 rooms:

- ~85,000 hotels globally

Room type inventory table rows:

- Hotels: ~85,000
- Average room types per hotel: 10
- Horizon: 2 years = 730 days

Rows:

- 85,000 × 10 × 730 ≈ 620M rows

Assuming each row is around 100 bytes:

- Storage ≈ 620M × 100 bytes ≈ 62 GB of raw data

Reality check:

With indexes (typically 2-3x table size), related tables (reservations, hotels, guests), you're looking at 500GB+ total.

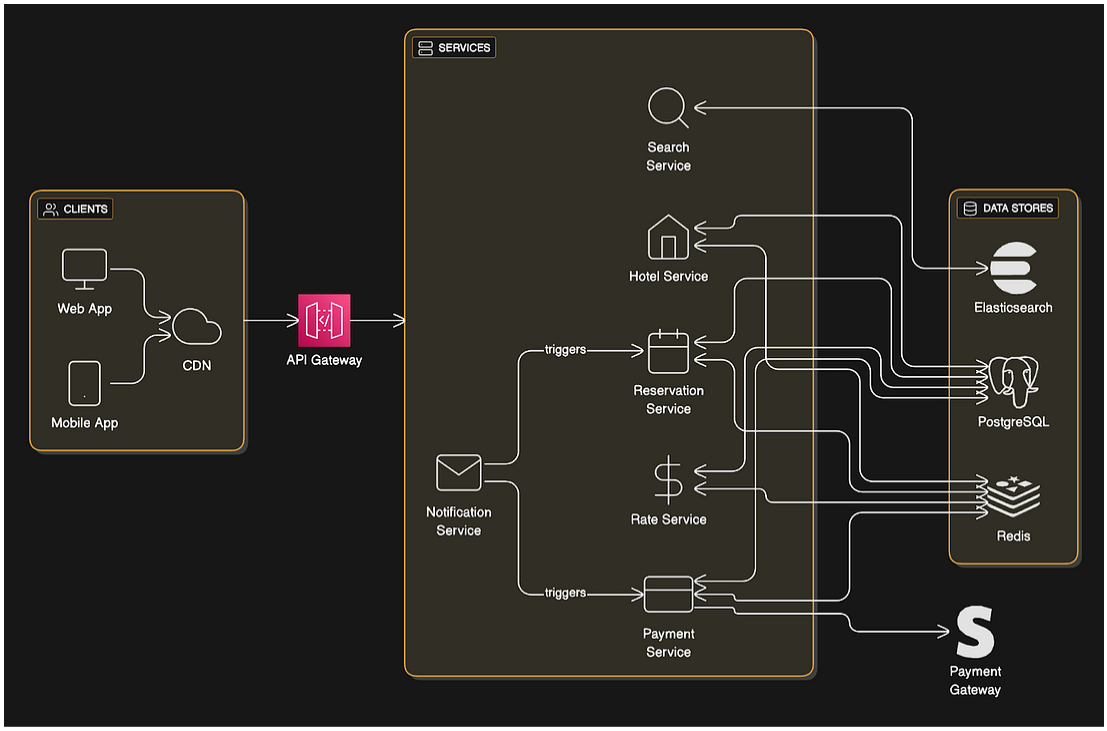
So you need a proper 'sharding and indexing strategy'.

High-Level Design

With the scale and data size understood, we can design an architecture that actually fits these numbers…

We're using a service-oriented architecture with clear boundaries, but keeping "tightly coupled" data on the same database.

Core Components



Client Layer

- Web and mobile apps
- CDN for static assets (images, JavaScript, hotel photos)

API Gateway

- Single entry point
- Authentication, rate limiting, request routing
- Load balancing across service instances

Services

- **Hotel Service**: Hotel and room information. Mostly static, heavily cacheable.
- **Rate Service**: Room pricing by date. Dynamic pricing based on demand.
- **Reservation Service**: Handles booking logic, inventory checks, and reservation creation/cancellation. This is where concurrency matters.
- **Payment Service**: Integrates with payment gateways. Updates the reservation status.
- **Notification Service**: Sends emails and push notifications.
- **Search Service**: Manages Elasticsearch[9] integration for hotel discovery.

Data Layer

- **PostgreSQL**: Primary transactional store. ACID guarantees[10].
- **Redis**: Cache for hot data.
- **Elasticsearch**: Search functionality (full-text, geospatial, filtering).

Why This Architecture?

Once services and boundaries are clear, the next step is defining how clients and services interact...

> At this scale, a pure monolith or a full microservices setup would be suboptimal.

A single monolith simplifies transactions but becomes hard to scale and deploy independently. Full microservices with separate databases introduce distributed transactions and consistency problems that don't exist at this traffic level.

This design uses service-level separation with shared transactional data for tightly coupled domains (reservations + inventory). It maintains deployment flexibility while preserving strong consistency where it matters most.

Each service owns its domain and scales independently.

Hotel Service might need 10 instances for search traffic, while Reservation Service only needs 3 since booking TPS is low.

Critical decision:

Reservation and inventory data live in the same database.

This lets us use local ACID transactions instead of distributed transactions. We have service-level separation at the application layer, but shared data for tightly coupled entities.

This is pragmatic.

Distributed transactions add massive complexity for minimal benefit here. The alternative (separate DBs with saga patterns or 2PC) is overkill when you can keep related data together.

Alternative:

Split reservations and inventory into separate databases and coordinate with Saga patterns[11] or Two-Phase Commit[12] (**2PC**).

2PC provides strong consistency but adds latency, blocking, and failure complexity. A coordinator failure can stall the system.

Saga patterns avoid locking but introduce eventual consistency[13], compensating actions, and complex failure handling. This is risky for user-facing booking flows where “eventually consistent” means angry users.

At ~17 write TPS, the added complexity is not justified… Local ACID transactions are simpler, safer, and faster.

API Design

RESTful APIs for all operations:

Hotel APIs (Admin)

- `GET /v1/hotels/{id}` - Get hotel details
- `POST /v1/hotels` - Add a new hotel (admin only)
- `PUT /v1/hotels/{id}` - Update hotel info (admin only)
- `DELETE /v1/hotels/{id}` - Remove hotel (admin only)

Room Type APIs (Admin)

- `GET /v1/hotels/{id}/rooms/{id}` - Get room type details
- `POST /v1/hotels/{id}/rooms` - Add room type (admin only)
- `PUT /v1/hotels/{id}/rooms/{id}` - Update room type (admin only)
- `DELETE /v1/hotels/{id}/rooms/{id}` - Delete room type (admin only)

Reservation APIs

- `GET /v1/reservations` - Get the user's reservation history
- `GET /v1/reservations/{id}` - Get specific reservation details
- `POST /v1/reservations` - Create a new reservation
- `DELETE /v1/reservations/{id}` - Cancel reservation

Search API

- `GET /v1/search`
- Parameters: location, checkIn, checkOut, guests, beds, priceRange
- Returns: available hotels and room types

Critical Detail for Reservations

`POST /v1/reservations` request includes an idempotency key[14]:

This `reservationId` is generated on the frontend and prevents double bookings when users click submit many times.

APIs define behavior, but the data model determines whether the system can enforce correctness.

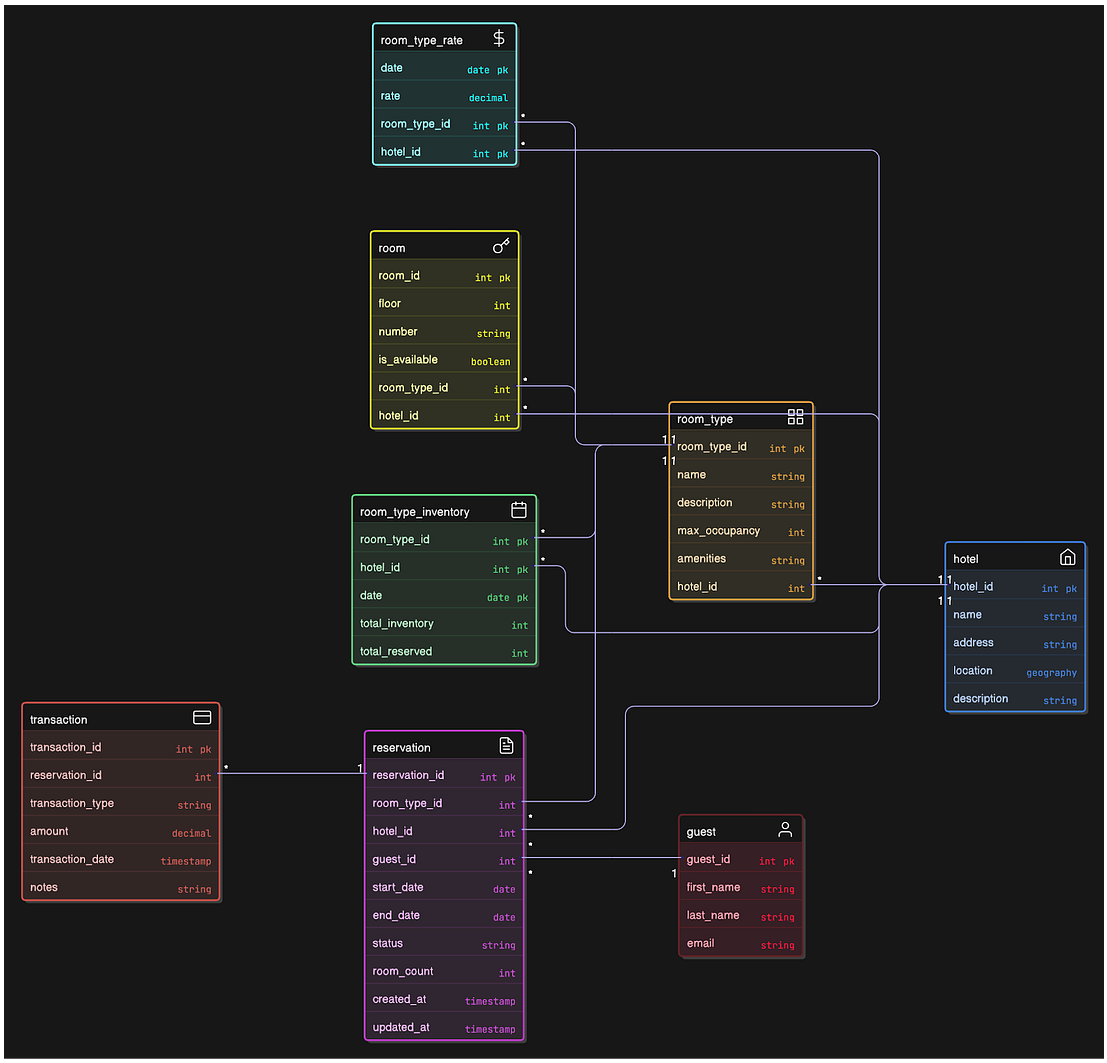
Let's keep going!

Data Model

> Here's a critical insight:
>
> "You book a room type (king bed, double queen), not a specific room. Room numbers get assigned at check-in."

Get this wrong, and your entire schema breaks.

Core Tables



We model hotels, room types, guests, reservations, and daily inventory/rates.

Core ideas:

- Hotels and room types form the catalog (what can be booked).
- Rooms exist physically, but bookings are made at the room type level (e.g., “Deluxe King”, not room #402).
- Inventory is tracked per hotel x room type x day, not per physical room. This enables:
    - bulk availability updates

    - scalable pricing and search
    - simple overbooking logic
- Rates (prices) are versioned daily per room type, similar to inventory.
- Reservations store a guest's booking (date range, quantity, status).
- Guests store personal details and link to reservations.
- Transactions: A common mistake is putting payment details directly into the `reservations` table. In a real-world system like Booking.com, a reservation is a "living" entity. If a guest changes their stay from 3 nights to 2, or adds a breakfast package later, you need to track the price difference without overwriting the original payment record.

Why This Schema Works

The `room_type_inventory` table has one row per (`hotel`, `room_type`, `date`) combination.

We pre-populate this for the next 2 years. And a daily `cron` job adds new dates as time moves forward.

Example data:

```
| hotel_id | room_type_id | date       | total_inventory | total_reserved |
|----------|--------------|------------|-----------------|----------------|
| 211      | 1001         | 2024-12-15 | 100             | 87             |
| 211      | 1001         | 2024-12-16 | 100             | 89             |
| 211      | 1001         | 2024-12-17 | 100             | 92             |
```

When checking availability from Dec 23 to Dec 27:

```sql
SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date BETWEEN '2024-12-23' AND '2024-12-27'
```

For each date in the range, verify:

```
(total_reserved + rooms_to_book) <= total_inventory
```

For 10% overbooking (configurable per hotel):

```
(total_reserved + rooms_to_book) <= (total_inventory * overbooking_factor)
```

Why Relational Database?

> With the schema in place, let's walk through the most critical path in the system: booking a room.

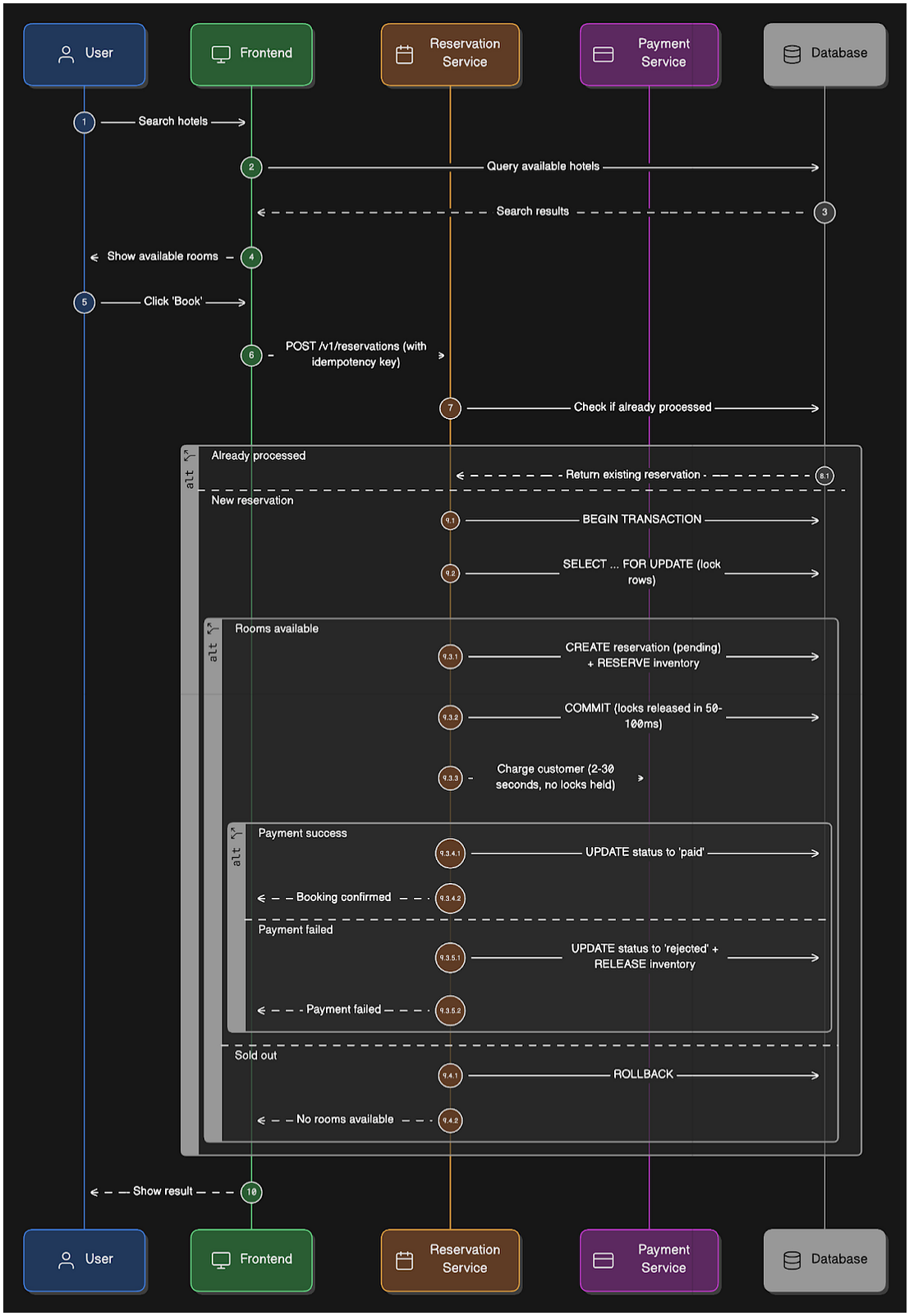
PostgreSQL or MySQL is the right choice here because of:

- **ACID guarantees**: Atomicity, Consistency, Isolation & Durability. Critical for preventing double bookings.
- **Read-heavy workload**: Relational databases handle reads well. We have 1,000 read QPS vs 17 write TPS.

- **Structured relationships**: Hotels, rooms, and reservations have well-defined relationships. Perfect for relational modeling.
- **Complex queries**: Need to join inventory with pricing, filter by dates, and aggregate totals. SQL handles this naturally.

The Reservation Flow

Here's what happens when a user books a room:


User
Frontend
Reservation Service
Payment Service
Database
1
Search hotels
2
Query available hotels
Search results
3
Show available rooms
4
5
Click 'Book'
6
POST /v1/reservations (with idempotency key)
7
Check if already processed
alt
Already processed
Return existing reservation
8.1
New reservation
9.1
BEGIN TRANSACTION
9.2
SELECT ... FOR UPDATE (lock rows)
alt
Rooms available
9.3.1
CREATE reservation (pending) + RESERVE inventory
9.3.2
COMMIT (locks released in 50-100ms)
9.3.3
Charge customer (2-30 seconds, no locks held)
alt
Payment success
9.3.4.1
UPDATE status to 'paid'
Booking confirmed
9.3.4.2
Payment failed
9.3.5.1
UPDATE status to 'rejected' + RELEASE inventory
Payment failed
9.3.5.2
Sold out
9.4.1
ROLLBACK
No rooms available
9.4.2
Show result
10
User
Frontend
Reservation Service
Payment Service
Database

1. **User searches for hotels:** Request hits Search Service, which queries Elasticsearch for matching hotels based on location, amenities, and price range.
2. **Check real-time availability:** For returned hotels, query PostgreSQL `room_type_inventory` table for actual availability during requested dates.
3. **Display results:** Combine search results with pricing from `room_type_rate` table. Show available room types with total price.
4. **User selects hotel and room type:** Frontend generates a unique `reservationId` (UUID) as an idempotency key.
5. **User clicks 'Book Now':** Frontend sends `POST /v1/reservations` with reservationId, room details, and dates.
6. **Idempotency check:** Check PostgreSQL reservation table for existing `reservationId`. If it exists, return the existing result (prevents duplicate clicks). If not, proceed.
7. **Inventory validation with transaction:** Begin a transaction and lock the inventory rows:

```sql
BEGIN;

SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date BETWEEN '2024-12-15' AND '2024-12-17'
FOR UPDATE;

-- Check in app: total_reserved + rooms_to_book <=
total_inventory

INSERT INTO reservations (
  reservation_id, hotel_id, room_type_id, guest_id,
  start_date, end_date, status, room_count, expires_at
) VALUES (
  ..., 211, 1001, ..., '2024-12-15', '2024-12-17',
  'pending', ..., NOW() + INTERVAL '15 minutes'
);

UPDATE room_type_inventory
SET total_reserved = total_reserved + rooms_to_book
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date BETWEEN '2024-12-15' AND '2024-12-17';

COMMIT;
```

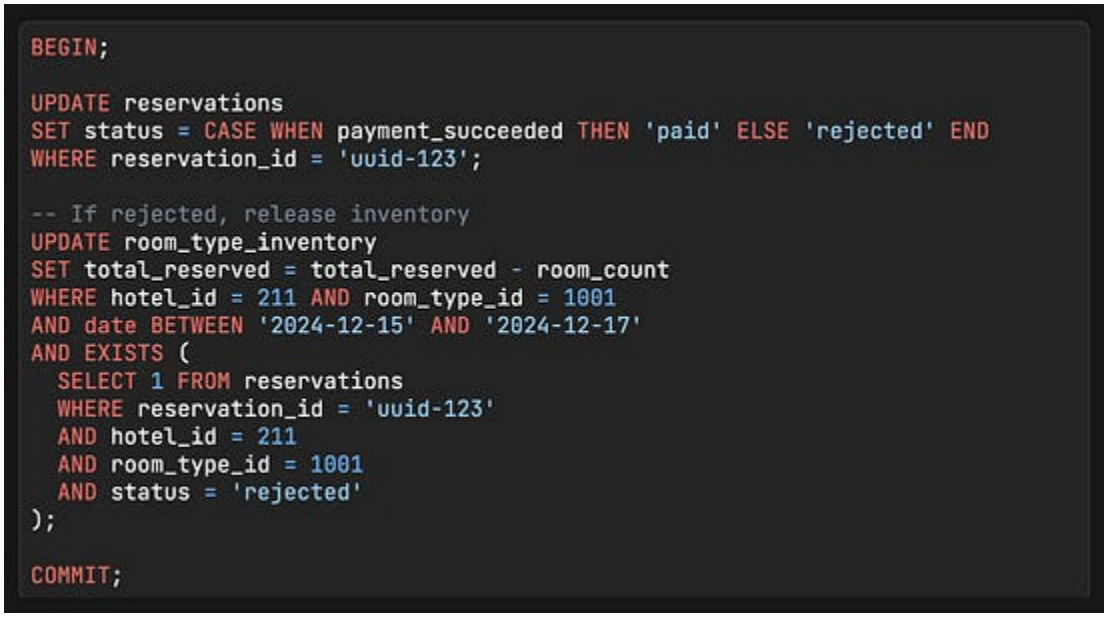
Transaction ends here, and the database locks are released. So the entire transaction takes 50-100ms. We never hold database locks during external API calls, such as payment processing.

8. **Process payment (outside transaction):** After locks are released, call Payment Service. This happens outside the database transaction:

```
payment_result = payment_service.charge(reservation_id='uuid-123',
amount=total_price, timeout=30)
```

This can take 2-30 seconds. No database locks are held during this time, so other bookings can proceed without waiting.

9. **Update reservation status:** Once payment completes, update the reservation in a separate, fast transaction:



```
BEGIN;

UPDATE reservations
SET status = CASE WHEN payment_succeeded THEN 'paid' ELSE 'rejected' END
WHERE reservation_id = 'uuid-123';

-- If rejected, release inventory
UPDATE room_type_inventory
SET total_reserved = total_reserved - room_count
WHERE hotel_id = 211 AND room_type_id = 1001
AND date BETWEEN '2024-12-15' AND '2024-12-17'
AND EXISTS (
  SELECT 1 FROM reservations
  WHERE reservation_id = 'uuid-123'
  AND hotel_id = 211
  AND room_type_id = 1001
  AND status = 'rejected'
);

COMMIT;
```

10. **Send notification:** Publish event to Kafka. Notification Service consumes and sends a confirmation email.
11. **Cleanup:** Background job checks for pending reservations older than 15 minutes. Auto-cancels and releases inventory.

Inventory validation with transaction:

Handling Concurrency

This is where most candidates fail...

Concurrency bugs happen even at READ COMMITTED isolation.

READ COMMITTED prevents dirty reads, but it still allows lost updates, write skew, and two transactions passing the same availability check.

NOTE: PostgreSQL's default isolation level is READ COMMITTED, which means you only see data after other transactions commit. But there's a

gap between when you READ (check availability) and WRITE (book the room). In that gap, another transaction can commit a booking. Both transactions read "99 rooms booked" at the same time, both think there's 1 room left, and both book it. This is a race condition.

So **READ COMMITTED by itself is unsafe** for bookings.

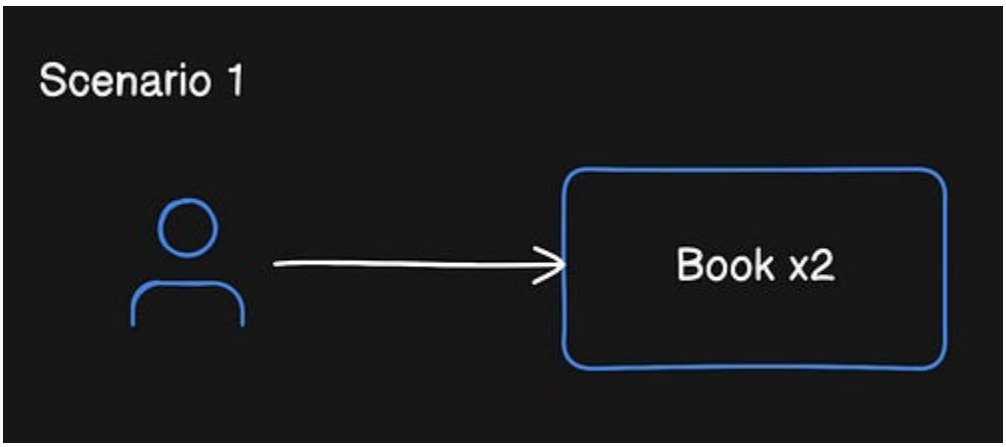
Two transactions can both read valid data, pass checks, and then overwrite each other. That's exactly how double bookings happen...

Let's break down the actual problem and solutions:

The Double Booking Problem

Two scenarios:

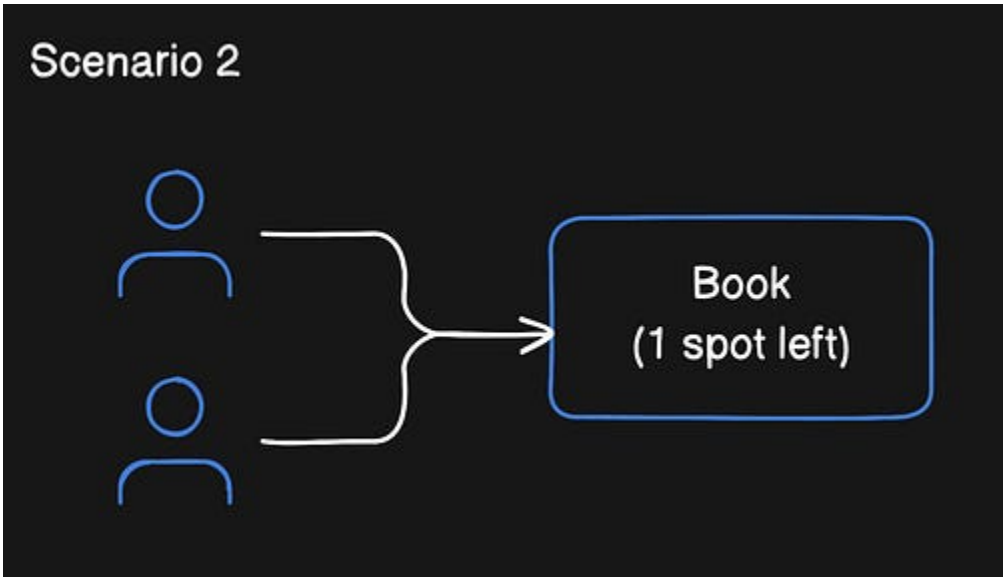
Scenario 1: Same user clicks 'Book' twice



Solution:

- Idempotency key (reservationId).
- Store in the database as the primary key.
- Second request gets duplicate key error; return existing reservation.

Scenario 2: Two users book the last room simultaneously



This is a hard one. Without proper concurrency control, this is what could happen:

```
Time 1: User A reads: total_reserved = 99, total_inventory = 100

Time 2: User B reads: total_reserved = 99, total_inventory = 100

Time 3: User A checks: (99 + 1) <= 100 ✓

Time 4: User B checks: (99 + 1) <= 100 ✓

Time 5: User A updates: total_reserved = 100 ✓

Time 6: User B updates: total_reserved = 100 ✓
```

Result: 2 bookings for 1 room.

Solution Comparison

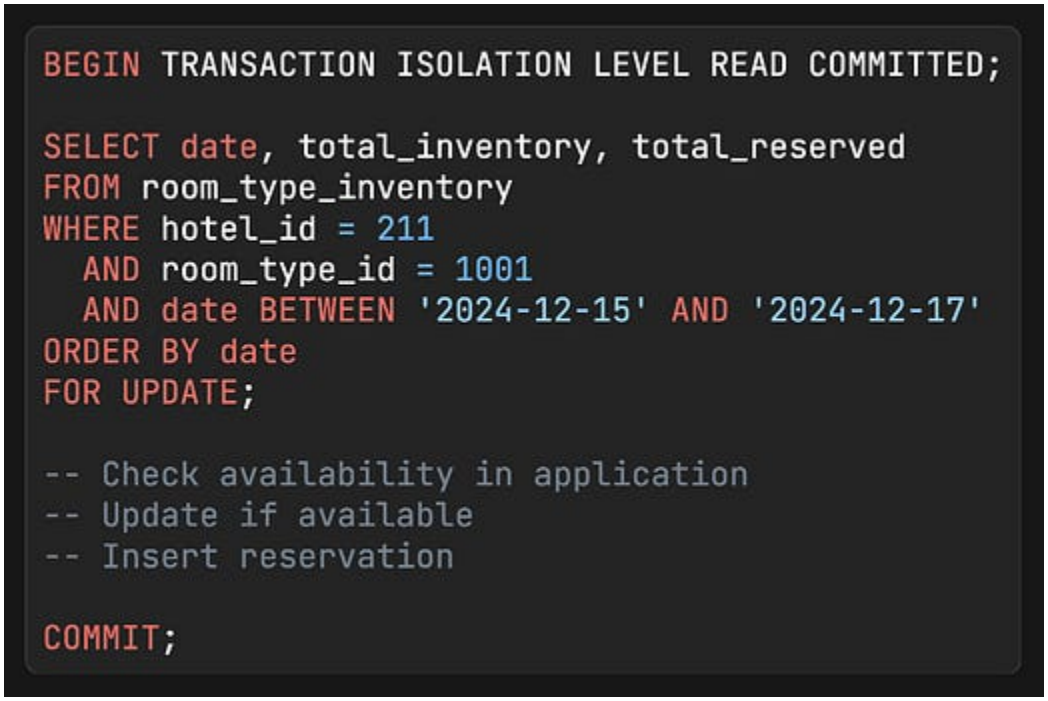
At 17 bookings per second with 85,000 hotels, each hotel sees a booking once every ~84 minutes on average.

This means, even during peak times (holidays, events), most hotels see low contention. It changes our approach...

Approach 1: READ COMMITTED Isolation + Row Locking

Use PostgreSQL's `READ COMMITTED` isolation level with `SELECT FOR UPDATE`:

`SELECT FOR UPDATE` is used to lock the inventory rows while we're checking and booking them. So that other users have to wait for their turn.



```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date BETWEEN '2024-12-15' AND '2024-12-17'
ORDER BY date
FOR UPDATE;

-- Check availability in application
-- Update if available
-- Insert reservation

COMMIT;
```

How it works:

- `SELECT FOR UPDATE` acquires exclusive row locks
- Other transactions are blocked until the lock is released

- READ COMMITTED is sufficient since `FOR UPDATE` handles the concurrency

Why not SERIALIZABLE?

Using `SERIALIZABLE` isolation with FOR UPDATE is redundant.

SERIALIZABLE already prevents the anomalies that FOR UPDATE protects against. The combination works, but adds unnecessary overhead.

READ COMMITTED + FOR UPDATE is simpler and performs better for this use case.

Pros:

- Simple to implement and reason about
- Guarantees no conflicts
- Works well at low to medium contention
- No application-level retry logic needed

Cons:

- Transactions block each other (but at 17 TPS globally, this is fine)
- Potential deadlocks if locking multiple resources in different orders
- Doesn't scale to thousands of concurrent writes to the same resource

Deadlock prevention:

- Always lock rows in consistent order: `ORDER BY hotel_id, room_type_id, date`
- Set `lock_timeout = '5s'` to prevent indefinite waits
- Keep transactions short

When to use this approach:

This is the recommended default for hotel reservations.

The write volume (17 TPS globally, ~1 booking per hotel per 84 minutes) is low enough that blocking isn't a problem. The simplicity and correctness guarantees make it the right choice unless we have proven high-contention issues.

Alternatives to this approach

Some teams try to avoid locking entirely.

Although these approaches appear attractive, they fail under concurrency...

1 Application-level mutex (in-memory lock):

Never use this for distributed systems.

Works only on a single server instance. The moment you scale horizontally to multiple servers, two servers both think they have the lock, and you get double bookings.

2 Redis distributed locks:

This is used in case you need coordination across microservices with separate databases.

We don't use it here because it adds network hops and failure modes. If Redis goes down, bookings stop. Lock expiry bugs cause both overbooking (expires too early, two clients get it) and underbooking (expires too late, inventory stuck).

PostgreSQL row locks are simpler and more reliable when you're already using PostgreSQL.

3 CHECK constraints only[15]:

We use this as a safety layer on top of locking, not as the primary mechanism.

It catches bugs in your application logic. But it's not sufficient alone because constraints are validated after writes, not during read-modify-write races. Two transactions can both read `total_reserved = 99`, both increment to 100, and both pass the constraint check. You still get double booking.

4 Queue-based booking:

Use this in high-contention scenarios like concert tickets, where thousands of users compete for the same item simultaneously.

A Queue ensures strict ordering. We don't use it here because hotel bookings have low contention (17 TPS globally, about 1 booking per hotel per 84 minutes). Queues add latency, complexity, and failure modes. Database row locking is simple and fast enough.

The bottom line:

These approaches either reduce correctness or add unnecessary complexity compared to database row locking. Stick with Approach 1 unless you have a proven need for something else.

Approach 2: Optimistic Locking with Versioning

Add a version column and check it during update[16]:

```sql
-- Add version column
ALTER TABLE room_type_inventory
ADD COLUMN version INTEGER DEFAULT 0;

-- Read with version
SELECT date, total_inventory, total_reserved, version
FROM room_type_inventory
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date BETWEEN '2024-12-15' AND '2024-12-17';

-- Store versions in application: {date: version}
-- expected_dates = 3

BEGIN;

-- Update all dates atomically, checking all versions
UPDATE room_type_inventory
SET total_reserved = total_reserved + 1,
    version = version + 1
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date = ANY(ARRAY['2024-12-15', '2024-12-16', '2024-12-17'])
  AND (
    (date = '2024-12-15' AND version = 5) OR
    (date = '2024-12-16' AND version = 7) OR
    (date = '2024-12-17' AND version = 3)
  );

-- Check rows affected
-- If affected_rows < expected_dates, rollback and retry
IF affected_rows < 3 THEN
  ROLLBACK;
  -- Retry from the beginning (re-read versions)
ELSE
  COMMIT;
END IF;
```

How it works:

- Read all dates with their versions

- Attempt to update all rows in one statement
- If ANY row's version changed (`affected_rows < expected`), another transaction modified it
- Rollback entire transaction and retry from scratch
- This ensures all-or-nothing updates across multiple dates

Pros:

- No locks held during transaction
- Better throughput under low contention
- No deadlock risk

Cons:

- Requires retry logic in the application
- Under high contention, many retries = poor performance
- Users see "*room available*" then get an error after clicking book
- Complex to update multiple date rows atomically

When to use:

- When you have very low contention and want maximum throughput.
- Not ideal for user-facing operations where retry = bad UX.

Approach 3: Atomic Decrement with Database Constraint

Let the database enforce availability[17]:

```sql
-- Add check constraint as safety net
ALTER TABLE room_type_inventory
ADD CONSTRAINT check_room_availability
CHECK (total_reserved <= total_inventory);

-- Update within transaction with row locking
BEGIN;

SELECT 1 FROM room_type_inventory
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date BETWEEN '2024-12-15' AND '2024-12-17'
ORDER BY date
FOR UPDATE;

UPDATE room_type_inventory
SET total_reserved = total_reserved + 1
WHERE hotel_id = 211
  AND room_type_id = 1001
  AND date BETWEEN '2024-12-15' AND '2024-12-17'
  AND total_reserved < total_inventory;

-- Check rows affected
-- If < expected dates, some dates are full

INSERT INTO reservations (...);

COMMIT;
```

IMPORTANT:

The WHERE clause total_reserved < total_inventory is critical.

Without it, two concurrent transactions can both increment and both pass CHECK constraint.

Pros:

- Simple to implement
- Database enforces the business rule
- No explicit locking in application code

Cons:

- CHECK constraint alone doesn't prevent race conditions
- Still need WHERE clause check or transaction isolation
- Harder to debug than application logic
- Not all databases handle CHECK constraints well under concurrency

When to use:

As an additional safety layer on top of other approaches, not as the primary mechanism.

Recommendation

> For hotel reservations at this scale, READ COMMITTED + SELECT FOR UPDATE is sufficient.

Here's the reasoning:

- FOR UPDATE prevents the race condition by serializing access to specific rows
- READ COMMITTED is PostgreSQL's default and works fine here
- SERIALIZABLE adds overhead without benefit when we're already using explicit locks
- Simpler to reason about: *"I lock these rows, modify them, release locks."*

SERIALIZABLE is not needed because:

- We use explicit row-level locking (FOR UPDATE)
- We touch only one table in the critical section
- No phantom reads possible (we lock by primary key)
- Simpler and performs identically at our scale

Scaling the System

At 1,000 read QPS and 17 write TPS, here's how to handle it:

Database Sharding

Shard[18] by `hotel_id` when a single database can't handle the load.

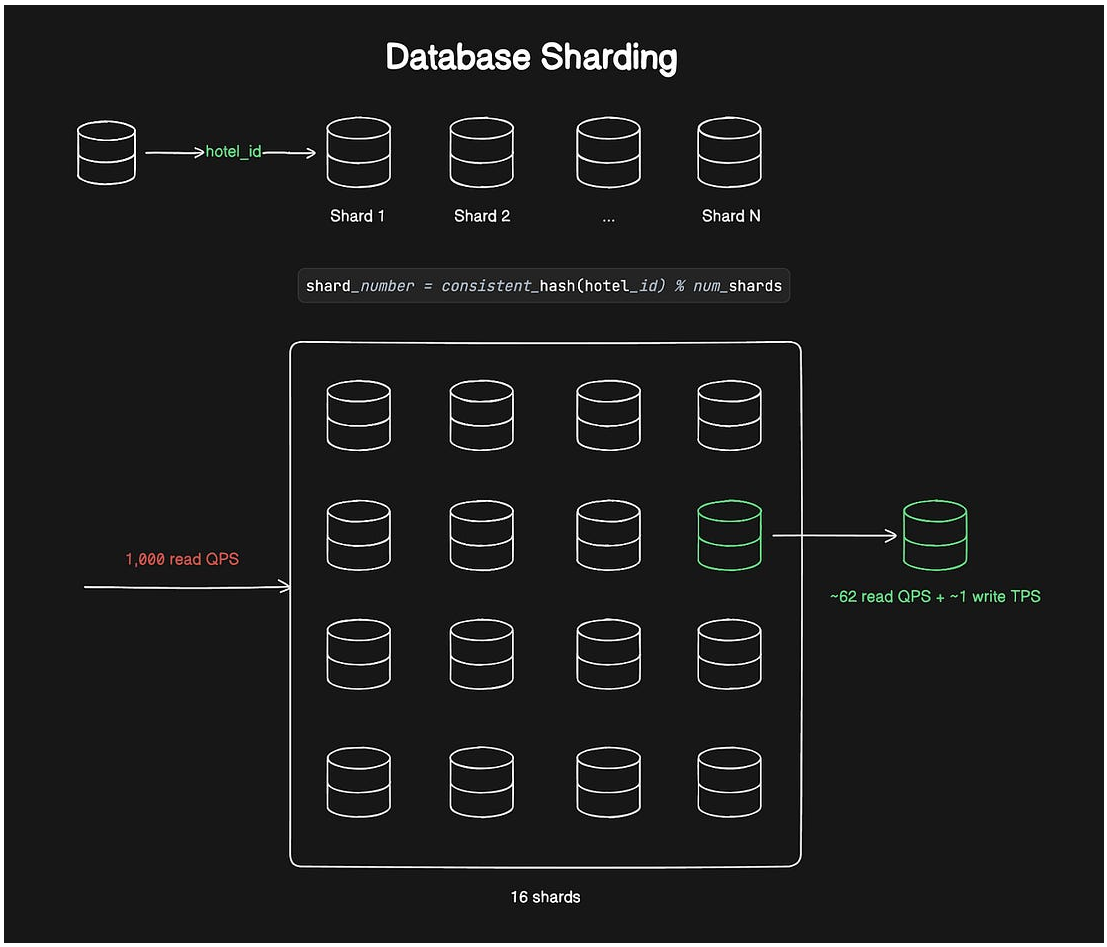
Why `hotel_id`?

- Most queries filter by hotel first
- All data of one hotel stay on the same shard (no cross-shard joins)
- Enables local transactions within the shard

Sharding Formula

For initial deployment, we can use simple modulo hashing:

```
shard_number = hash(hotel_id) % num_shards
```



When we need to add shards later, we can migrate to consistent hashing to minimize data movement.

With modulo hashing, adding a 17th shard requires reshuffling almost all data.

Consistent hashing[19] moves only about 1/17 of the data (to the new shard). It uses a hash ring where both servers and keys map to points on a circle. Each key goes to the next server on the ring, clockwise.

So adding a server only affects keys between it and the previous server.

For hotel reservations, modulo hashing is fine initially because:

- Shard count is relatively stable (you don't add shards weekly)

- Resharding can be done during low-traffic hours
- It's simpler to implement and debug

However, if you expect frequent shard additions, use consistent hashing from the start.

Challenge with search:

When users search by location without knowing `hotel_id`, you have two options:

1. **Scatter-gather**: Query all shards, merge results. Works but is slower.
2. **Routing table**: Maintain a mapping of `location → hotel_ids` in a separate service. Query this first, then route to specific shards.

We use option #2:

The routing table is small (hotels don't move), heavily cached, and efficient for sharded queries.

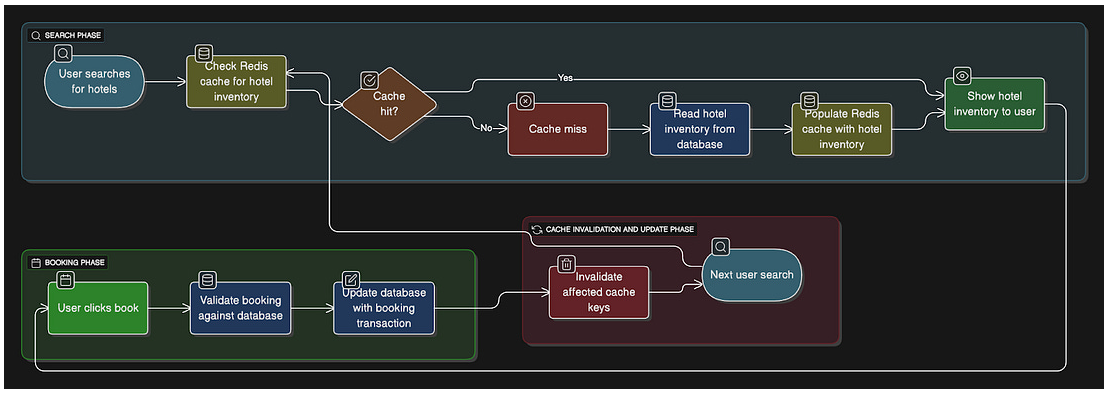
Caching Strategy

Cache hot inventory data in Redis with smart invalidation.

Cache structure:

```
Key: inventory:{hotel_id}:{room_type_id}:{date}
Value: {total_inventory: 100, total_reserved: 87}
TTL: 5 minutes
```

Cache flow:



1. User searches hotels → Check Redis cache for inventory
2. If cache hit → Show results to user
3. User clicks “Book” → Always validate against database (source of truth)
4. Update the database with the transaction
5. Immediately invalidate affected cache keys
6. Next search will cache miss, read from DB, populate cache

Why immediate cache invalidation[20] instead of CDC?

Async CDC[21] (Debezium[22]) adds lag (milliseconds to seconds).

During that lag:

- User A books the last room (DB updated)
- User B checks cache (still shows available)
- User B tries to book (fails at DB validation)
- This is Bad UX

Immediate invalidation on writes keeps the cache miss rate low while avoiding stale reads during the booking flow.

CDC Use Cases

CDC works great for:

- Syncing to Elasticsearch (eventual consistency fine)
- Analytics databases (lag acceptable)
- Audit logs (non-critical path)

But don't use it for transactional cache invalidation.

Search Optimization with Elasticsearch

Elasticsearch handles:

- Full-text search[23] of hotel names and descriptions
- Geospatial queries[24] (hotels within 5km of the location)
- Filtering by amenities, price range, and ratings
- Fast aggregations

Search flow:

1. User searches "hotels in Paris with Wi-Fi, price < $200"

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "location": "Paris" } },
        { "range": { "base_price": { "lte": 200 } } }
      ],
      "filter": [
        { "term": { "amenities": "wifi" } }
      ]
    }
  }
}
```

Query Elasticsearch

2. Elasticsearch returns hotel IDs and room type IDs
3. Query PostgreSQL for real-time availability for those IDs
4. Combine the results and return to the user

How to Keep Elasticsearch in sync:

Use the CDC pipeline:

- Debezium monitors the PostgreSQL transaction log
- Changes published to Kafka
- Elasticsearch consumer updates indexes

Eventual consistency is fine here. Because if Elasticsearch shows a hotel that's fully booked, the availability check catches it.

Denormalization strategy:

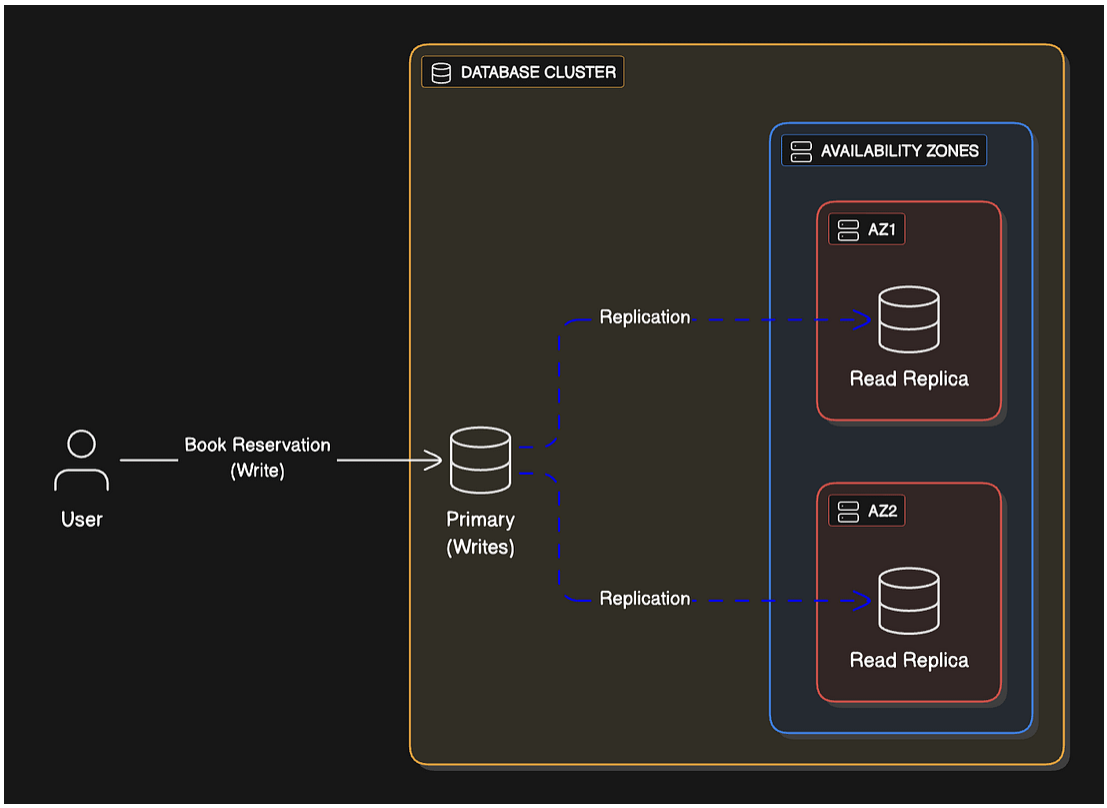
Elasticsearch documents include denormalized data:

```
{
  "hotel_id": "211",
  "name": "Grand Hotel Paris",
  "address": "123 Rue...",
  "city": "Paris",
  "location": {"lat": 48.8566, "lon": 2.3522},
  "room_types": [
    {"id": "1001", "name": "Deluxe King", "base_price": 150},
    {"id": "1002", "name": "Suite", "base_price": 300}
  ],
  "amenities": ["wifi", "pool", "gym"],
  "rating": 4.5,
  "review_count": 1523
}
```


When a hotel or room type changes, re-index the entire hotel document... This is async and eventually consistent.

Database Replication

- Primary database (writes)
- 2+ read replicas[25] (reads)
- Replicas in different availability zones



Read-your-writes consistency:

After a user books, they immediately check *"My Reservations"*. So if we read from a replica, they might not see it yet because of replication lag[26].

Solution:

```
function getUserReservations(userId, recentWrite = false) {
  const shouldUsePrimary =
    recentWrite || hasRecentWriteInSession(userId);

  // Read from primary for ~10s after a write, otherwise replica
  const db = shouldUsePrimary
    ? getPrimaryConnection()
    : getReplicaConnection();

  return db.query(
    "SELECT * FROM reservations WHERE guest_id = ?",
    [userId]
  );
}
```

Track recent writes in Redis:

Key: user:write:{user_id}

Value: timestamp

TTL: 10 seconds

So when a booking request comes in:

1. **After writing to the primary database**, the app stores a tracking key in Redis
2. **When the user requests "My Reservations"**, the app checks Redis for user:write:{user_id}
3. **If the key exists and the timestamp is recent** (within the last 10 seconds), route the read to the primary database (guarantees they see their booking)
4. **If the key is missing or expired**, route the read to replicas (replication has caught up by now)

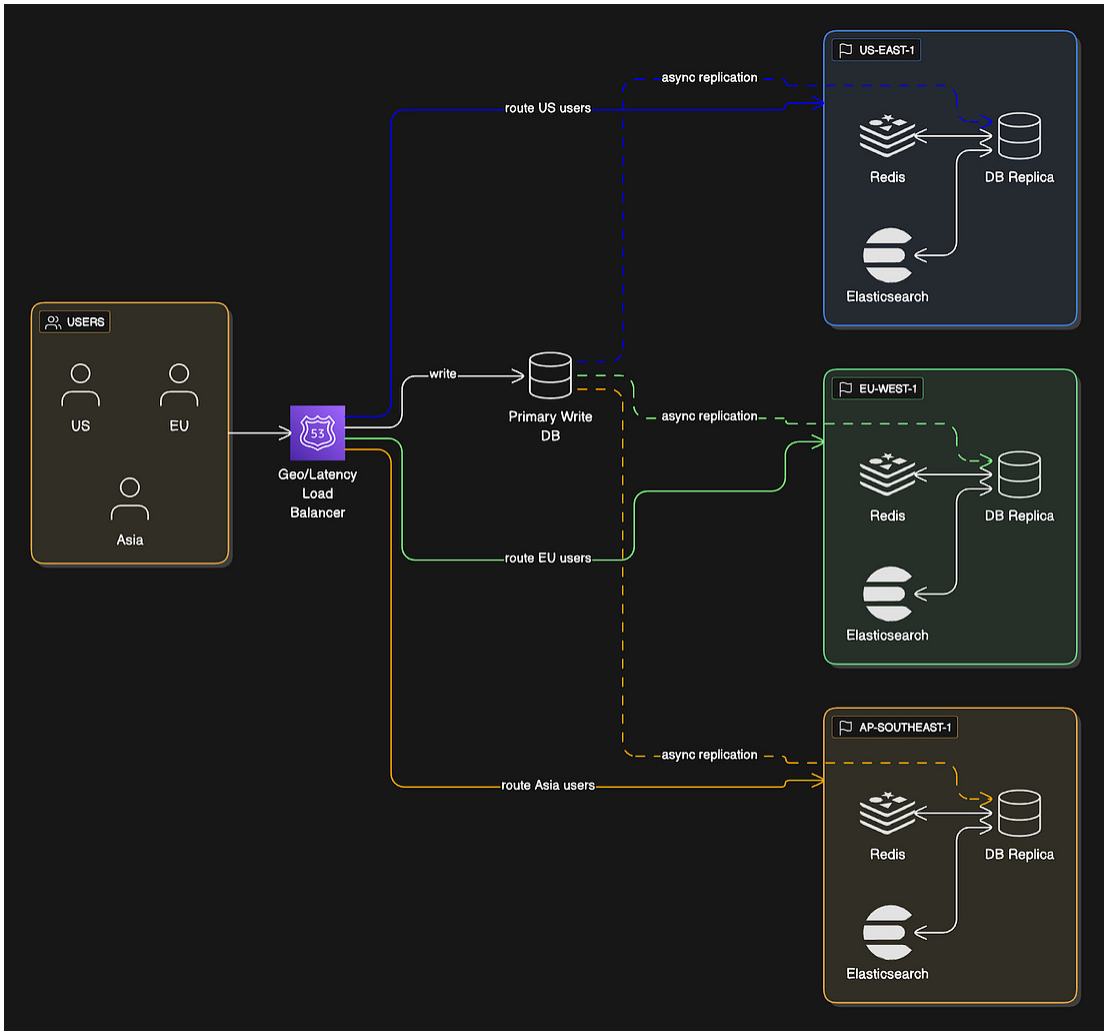
This ensures users always see their own writes while still using fast replicas for 99% of read traffic.

Load Balancing

Use geographic load balancing.

- Users in US → us-east-1 region
- Users in EU → eu-west-1 region
- Users in Asia → ap-southeast-1 region

Route based on lowest latency...



Each region has:

- Full database replica (async replication)

- Local Redis cache
- Local Elasticsearch cluster

Writes go to the primary region and get synced globally. While Reads are local.

Tradeoff:

Potential for stale reads across regions.

If the user books in the US and then immediately checks in the EU via a different network (for example, a VPN), they might not see it yet. This is acceptable in this use case.

Handling Failures and Edge Cases

Systems fail… Here's how to handle it:

Payment Failures

Problem:

Payment fails after the inventory is decremented.

Wrong approach:

1. Decrement inventory
2. Create a reservation (pending)
3. Charge card (declined)
4. Inventory stuck as reserved

CORRECT approach:

Use a reservation status state machine with automatic cleanup:

```
BEGIN;
-- Lock and reserve inventory
UPDATE room_type_inventory ...
-- Create reservation
INSERT INTO reservations (
 reservation_id, status, created_at, expires_at
) VALUES (
 'uuid-123', 'pending', NOW(), NOW() + INTERVAL '15 minutes'
);
COMMIT;
-- Attempt payment (async)
payment_result = payment_service.charge(...)
-- Update based on result
UPDATE reservations
SET status = CASE
 WHEN payment_result = 'success' THEN 'paid'
 ELSE 'rejected'
END
WHERE reservation_id = 'uuid-123';
-- If rejected, release inventory
IF status = 'rejected':
 UPDATE room_type_inventory
 SET total_reserved = total_reserved - room_count
 WHERE ...
```

Background cleanup job[27]:

Every minute, find expired pending reservations.

```sql
-- Find expired pending reservations
SELECT reservation_id, hotel_id, room_type_id,
 start_date, end_date, room_count
FROM reservations
WHERE status = 'pending'
 AND expires_at < NOW();

-- For each:
-- 1. Update status to 'expired'
-- 2. Decrement inventory
-- 3. Send notification to user
```

This prevents inventory leaks if the payment service crashes or the user abandons checkout.

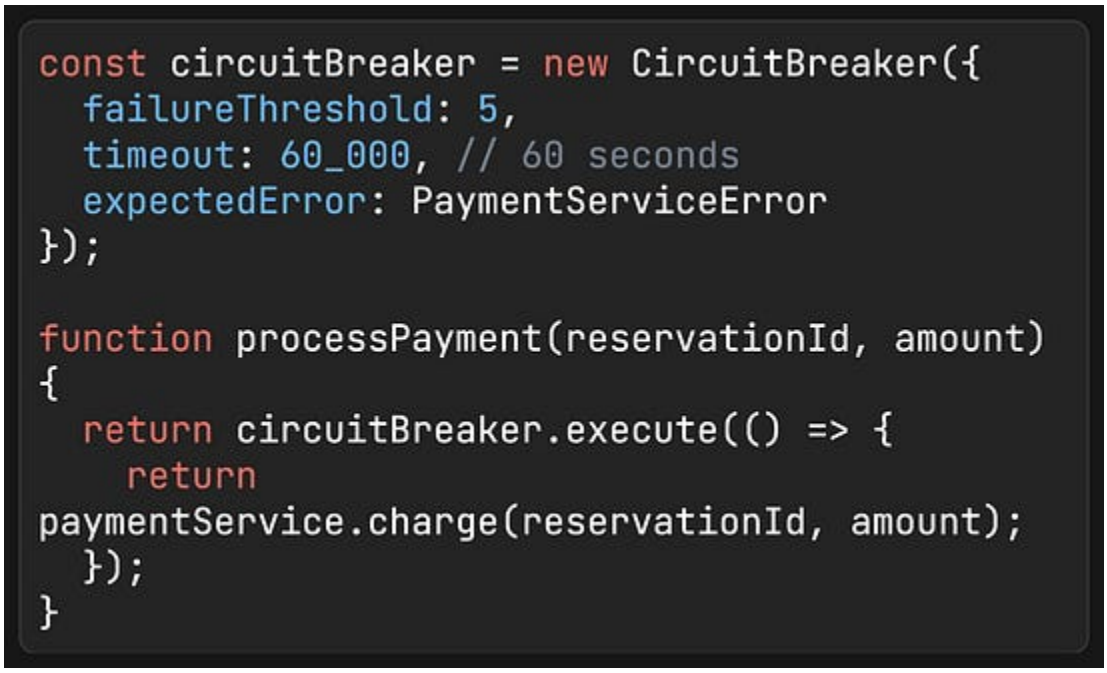
Service Failures

Scenario:

Reservation Service crashes mid-transaction.

Solutions:

1. **Database transactions**: PostgreSQL automatically rolls back uncommitted transactions. No partial state persisted.
2. **Idempotency**: Client retries with the same `reservationId`. Database rejects a duplicate key and returns the existing reservation.
3. **Circuit breakers**[28]: If the Payment Service is down, don't hammer it. Open circuit, fail fast, queue in Kafka for later processing.



```
const circuitBreaker = new CircuitBreaker({
  failureThreshold: 5,
  timeout: 60_000, // 60 seconds
  expectedError: PaymentServiceError
});

function processPayment(reservationId, amount)
{
  return circuitBreaker.execute(() => {
    return
paymentService.charge(reservationId, amount);
  });
}
```

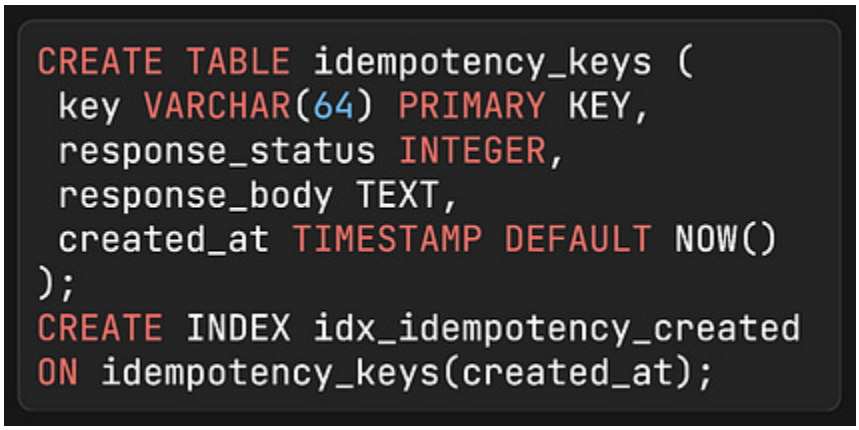
4. **Distributed tracing**: Use Jaeger to trace requests across services. When something fails, see exactly where.

Idempotency Implementation

Store idempotency keys in the database for durability:

When a user submits a booking request, we generate a unique idempotency key (UUID) on the frontend.

This key is stored in a dedicated table along with the response. If the user clicks "Book" twice (because of a slow network or impatience), the second request with the same key returns the cached response instead of creating a duplicate booking.



```
CREATE TABLE idempotency_keys (
 key VARCHAR(64) PRIMARY KEY,
 response_status INTEGER,
 response_body TEXT,
 created_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_idempotency_created
ON idempotency_keys(created_at);
```

The idempotency_keys table stores the key as the primary key, so any duplicate insertion automatically fails.

We also cache frequently accessed keys in Redis for faster lookups (check Redis first, fall back to the database if a cache miss). Old keys are automatically cleaned up after 7 days since they're no longer needed.

Also cache the keys in Redis for fast lookup:

> Key: idempotency:{key}
> Value: {status: 200, reservation_id: “res-123”}
> TTL: 24 hours

Request flow:

```
async function createReservation(req) {
  const idempotencyKey = req.headers["Idempotency-Key"];
  const redisKey = `idempotency:${idempotencyKey}`;

  // 1) Fast path: check Redis cache
  const cached = await redis.get(redisKey);
  if (cached) return cached;

  // 2) Check DB if Redis missed / expired
  const dbRecord = await db.query(
    "SELECT * FROM idempotency_keys WHERE key = ?",
    [idempotencyKey]
  );
  if (dbRecord.length > 0) {
    await redis.setex(redisKey, 86400, dbRecord[0]); // cache for next time
    return dbRecord[0];
  }

  // 3) Process normally
  const response = await processReservation(req);

  // 4) Persist idempotency result
  await db.execute(
    "INSERT INTO idempotency_keys (key, response_status, response_body) VALUES (?, ?, ?)",
    [idempotencyKey, response.status, response.body]
  );
  await redis.setex(redisKey, 86400, response);

  return response;
}
```

Plus, clean up old keys periodically to free up storage space.

```
DELETE FROM idempotency_keys
WHERE created_at < NOW() - INTERVAL '7 days';
```

Cache Inconsistency

Problem:

Cache says the room is available, but DB says it's not.

Mitigation:

1. **Always validate against DB**: Cache is an optimization, not a source of truth.
2. **Invalidate on writes**: When inventory updates, immediately delete cache keys:

```
function updateInventory(hotelId, roomTypeId, dates) {
  // Update DB
  db.execute(
    "UPDATE room_type_inventory SET ... WHERE hotel_id = ? AND room_type_id = ?",
    [hotelId, roomTypeId]
  );

  // Invalidate cache
  const keys = dates.map(date =>
    `inventory:${hotelId}:${roomTypeId}:${date}`
  );

  redis.del(...keys); // remove multiple keys
}
```

3. **Short TTL**: 5-minute TTL limits the staleness window.
4. **Show optimistic UI**: After booking, show the reservation in the UI immediately. Don't wait for cache refresh.

The key insight:

Cache inconsistency causes the user to see availability that doesn't exist. But DB validation catches this. The user gets an error message, but the inventory remains correct. Annoying but not catastrophic.

Overbooking Edge Cases

Hotels allow 5-10% overbooking, expecting some cancellations. But what if everyone shows up?

This is a business problem, not a technical one.

Hotels have policies:

- Upgrade to a better room type (free)
- Book a room at a partner hotel (hotel pays)
- Compensation (free night, voucher)

Technical requirement: Enforce the overbooking cap accurately.

```sql
-- Add overbooking_factor to hotel config
ALTER TABLE hotels
ADD COLUMN overbooking_factor DECIMAL(3,2) DEFAULT 1.10;
-- Check availability with overbooking
SELECT date, total_inventory, total_reserved, overbooking_factor
FROM room_type_inventory
JOIN hotels USING (hotel_id)
WHERE hotel_id = 211
 AND room_type_id = 1001
 AND date BETWEEN '2024-12-15' AND '2024-12-17';
```

In application:

```
allowed = total_inventory * overbooking_factor

available = (total_reserved + rooms_to_book) <= allowed
```

Some hotels don't allow overbooking (set factor to 1.0).

The system enforces per hotel.

Regional Failover

For high availability across regions:

Setup:

- Primary region (writes + reads)
  - us-east-1
- Replica regions (reads only)
  - eu-west-1
  - ap-southeast-1

Failover strategy:

1. DNS (Route53) health checks detect primary failure
2. Promote a replica to the primary in another region
3. Update the DNS to route traffic to the new primary

Typical downtime: 2-5 minutes.

Challenge:

Newly promoted primary might be slightly behind. And this can cause booking conflicts.

Mitigation:

- `reservation_id` as primary key prevents duplicates
- Idempotency checks survive failover (stored in DB)
- Manual review of any conflicts post-failover
- Synchronous replication for critical tables (reservations, inventory) if you need zero data loss

Tradeoff:

Synchronous replication adds latency to writes. For most hotel bookings, async replication with occasional manual conflict resolution is acceptable.

Database Deadlocks

With row locking, deadlocks can occur.

> Transaction A: Locks date 2024-12-15, tries to lock 2024-12-16
> Transaction B: Locks date 2024-12-16, tries to lock 2024-12-15
>
> Now both transactions are waiting on each other forever.

Prevention:

- **Lock in consistent order**: Always `ORDER BY date`[29]
- **Set lock timeout**[30]: `SET lock_timeout = '5s';`
- **Retry on deadlock**[31]: PostgreSQL error code `40P01`

```
@retry(
retry=retry_if_exception_type(DeadlockException),
stop=stop_after_attempt(3),
wait=wait_exponential(multiplier=1, min=1, max=10)
)
```

- **Keep transactions short**[32]: Don't hold locks during external API calls (like payment).

At 17 TPS with proper ordering, deadlocks should be rare (< 0.1% of transactions).

Key Takeaways

Scope properly:

Ask about scale, payment timing, overbooking, and dynamic pricing upfront.

Understand the data model:

Reserve room types, not specific rooms.

The `room_type_inventory` table is the core. Pre-populate for future dates.

Choose the right database:

Relational databases (PostgreSQL, MySQL) for ACID guarantees. Critical for preventing double bookings.

Prevent double booking:

- Idempotency keys for duplicate clicks
- `READ COMMITTED` isolation with row locking for concurrent users
- At 17 TPS, blocking is fine

Scale smartly:

- Shard by hotel_id to keep related data together
- Cache inventory in Redis with immediate invalidation
- Use Elasticsearch for search, PostgreSQL for bookings

Handle failures gracefully:

- Pending reservations with TTL for payment failures
- Idempotency in DB + Redis for crash recovery
- Circuit breakers for cascading failures
- Background jobs for cleanup

Optimize for reads:

This is read-heavy (1,000 QPS searches vs 17 TPS bookings).

Cache aggressively, use read replicas, but always validate writes against the primary.

Keep it simple:

Don’t use distributed transactions.

Keep the reservation and inventory in the same database. Use local ACID transactions.

Know your scale:

At 17 TPS, you don't need extreme solutions. Pessimistic locking works fine. Optimize for correctness and simplicity first, performance second.

This is the level of depth that gets you senior offers. You're not just drawing boxes. You're explaining tradeoffs, defending decisions with math, and anticipating failures.

Practice explaining this out loud.

System design interviews test communication as much as technical depth. Walk through the flow, explain your choices, and show you can think through the consequences of your decisions.

👋 I'd like to thank Hayk for writing this newsletter!

- Plus, don't forget to join his **YouTube channel**.

It'll help you master the essential system design skills, not just in theory but in practice, and land senior developer roles.

Know someone who wants to learn system design? Consider gifting a subscription:

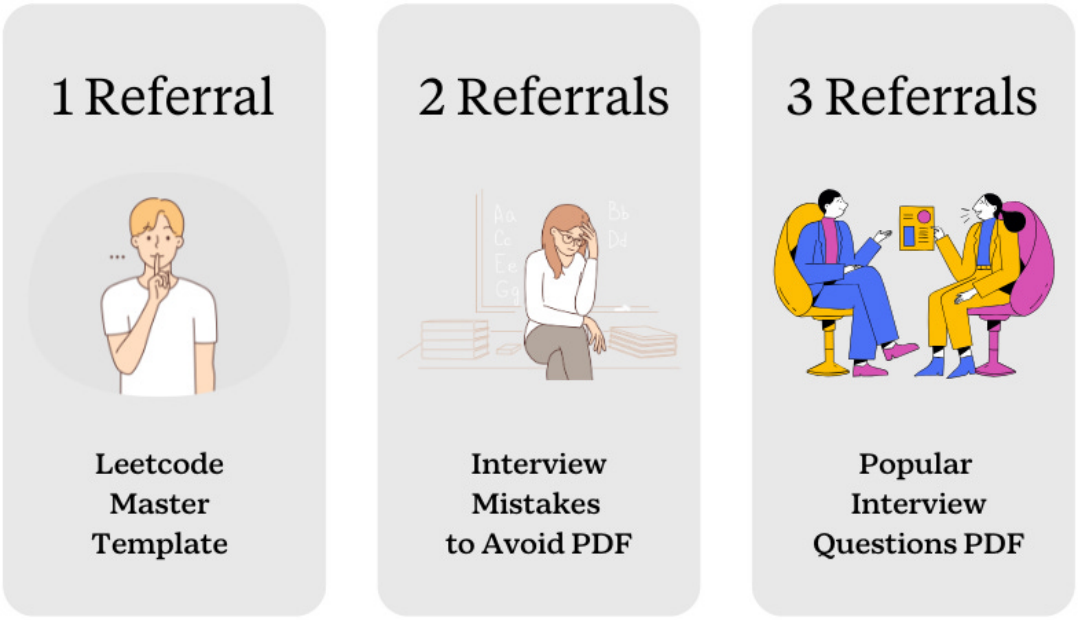
Give a gift subscription

👋 Find me on LinkedIn | Twitter | Threads | Instagram

Thank you for supporting this newsletter.

You are now 200,001+ readers strong, very close to 201k. Let's try to get 201k readers by 21 January. Consider sharing this post with your friends and get rewards.

Y'all are the best.



Share

1. **Concurrency** means multiple requests modifying or checking the same data simultaneously.

2. **Double booking** occurs when two users successfully book the same room for overlapping dates due to race conditions in the system.

3. **Back-of-the-envelope calculation** is a quick approximation used to reason about real-world scale during design interviews.

4. **Overbooking** is intentionally selling more rooms than available, assuming some guests will cancel or not show up.

5. **Inventory** in this context is the count of bookable room units per room type per date (not individual physical rooms).

6. **Room nights** represent one room occupied for one night. A 3-night stay in 2 rooms equals 6 room nights.

7. **TPS (Transactions Per Second)** measures write operations per second (e.g., new bookings).

8. **QPS (Queries Per Second)** measures the number of read operations per second (e.g., searching for availability or viewing hotels).

9. **ElasticSearch indexing** means storing denormalized, searchable copies of hotel data optimized for filtering, text search, and geolocation—not for booking transactions.

10. **ACID guarantees** are database properties ensuring correctness during transactions:

    - A: atomic (all or nothing),
    - C: consistent,
    - I: isolated from other operations,
    - D: durable (survives failures).

11. Saga pattern breaks a large transaction into smaller steps, each with a

compensating action to undo it if a later step fails.

12 Two-Phase Commit is a protocol where multiple systems first agree they can commit a transaction, and only then all commit it together to keep data consistent.

13 **Eventual consistency** means search results or cached availability might temporarily show stale data, but booking validation against the primary database prevents incorrect reservations.

14 **Idempotency key** ensures that a request executed multiple times (e.g., a user clicks “Book Now” twice) only results in one reservation.

15 It means that database CHECK constraints only verify the final value being written, but they do not protect the multi-step “read → decide → write” process from race conditions.

- A CHECK constraint is enforced after an update happens.
- If two transactions read the same value at the same time, they can both decide “this is valid” and both write updates.
- The CHECK constraint may still pass, even though the combined result is wrong.

Example:

- Two users see “1 room left.”
- Both decide it’s OK to book.
- Both update the counter.
- Each update individually satisfies the CHECK constraint.
- Result: overbooking!

Why the WHERE clause or locking is needed:

- A conditional `UPDATE` (for example: `WHERE available > 0`) makes the check atomic.

- `SELECT FOR UPDATE` locks the row so only one transaction can make the decision at a time.

16 Optimistic locking with versioning means you assume no one else is changing the data, but you check at update time and, if anything has changed, you cancel and retry.

17 This approach lets the database update the count only if rooms are still available, using one atomic update.

Why it matters:

A CHECK constraint only checks the final value, but the WHERE condition makes the check and update happen together, which avoids some races.

When to use it:

It's good as an extra safety net, but not strong enough by itself, so it should be combined with locking or proper isolation.

18 **Shard** is a partition of a database that stores a subset of data, such as all hotels assigned to a shard based on `hotel_id`.

19 **Consistent hashing** distributes data (e.g., hotel reservations) across shards to minimize data movement when servers are added or removed.

20 **Cache invalidation** means removing or updating stale values stored in fast-access memory (Redis) when the database changes.

21 CDC (Change Data Capture) is a technique for tracking and streaming changes made to a database, such as inserts, updates, and deletes.

In other words, CDC captures every committed data change and sends it to other systems so they can stay in sync without constantly querying the database.

22 **Debezium** is an open-source tool that reads your database's transaction log (the internal record of all changes) and publishes those changes as events.

When you update a hotel name in PostgreSQL, Debezium captures that change within milliseconds and publishes it to Kafka.

23 **Full-text search** allows searching natural text, such as hotel names or descriptions, beyond simple exact matches.

24 **Geospatial queries** return hotels near a location by calculating distance instead of exact string matching.

25 **Read replica** is a database copy that handles reads only, reducing load on the primary database.

26 **Replication lag** is the delay before data written to the primary database appears on replicas, potentially causing stale reads.

27 **Background cleanup job** is an automated scheduled task that fixes stuck or incomplete states (e.g., abandoned pending reservations).

28 **Circuit breaker** stops sending requests to a failing service (e.g., payments) after repeated errors, preventing cascading failures.

29 If every transaction locks rows in the same order (earliest date first), the circular wait cannot happen. Transaction A and B will both try to lock `2024-12-15` first, so one waits immediately instead of both partially locking different rows.

30 If a transaction waits too long for a lock, the database aborts it instead of letting it hang forever. This turns “infinite waiting” into a recoverable failure.

31 PostgreSQL automatically detects deadlocks and aborts one transaction. Catching error 40P01 and retrying lets the system recover transparently.

32 The shorter a transaction runs, the less time it holds locks. Not holding locks during slow operations (like payment APIs) dramatically reduces the chance of overlap and deadlocks.